#B Install an exception handler to signal invalid inputs to the caller as HTTP 400 errors.
#C Also handle exceptions from the JSON parser.
#D Return 404 Not Found for Dalesbred empty result exceptions.

Now the user gets an appropriate error if they supply invalid input:

```
$ curl -d '{"name":"test", "owner":"a really long username
[CA]that is more than 30 characters long"}'
[CA]http://localhost:4567/spaces -i
HTTP/1.1 400 Bad Request
Date: Fri, 01 Feb 2019 15:21:16 GMT
Content-Type: text/html;charset=utf-8
Transfer-Encoding: chunked
Server: Jetty(9.4.8.v20171121)
{"error": "java.lang.IllegalArgumentException: invalid
username: a really long username that is more than 30
characters long"}
```

## *EXERCISES*

3. Given the following code for processing binary data received from a user (as a `java.nio.ByteBuffer`):

```
int msgLen = buf.getInt();
byte[] msg = new byte[msgLen];
buf.get(msg);
```

a) Recalling from the start of section 2.5 that Java is a memory-safe language, what is the main vulnerability an attacker could exploit in this code?